CLEF SPORT

Base de Datos para Gestión Empresarial

Informe Técnico Final — Corte 3

Estudiante	David Salazar Lasso
Asignatura	Bases de Datos II
Docente	Helen Marcela Bustamante
Institución	Inst. Univ. Antonio José Camacho
Fecha	Junio 2026

3.1 — Integración con Aplicación Externa

3.1.1 Descripción General

El Corte 3 del proyecto Clef Sport consiste en conectar la base de datos MySQL con una aplicación de consola desarrollada en Python. La integración utiliza la librería mysql-connector-python y se comunica exclusivamente a través de procedimientos almacenados, sin ejecutar SQL directo desde el código de la aplicación.

La aplicación implementa tres operaciones fundamentales del negocio: consulta del historial de compras de un cliente por cédula, registro de una nueva venta con validación de stock, y generación de reportes de ventas por rango de fechas.

3.1.2 Configuración de Conexión

La conexión a la base de datos se centraliza en un diccionario de configuración al inicio del archivo app.py. Esto permite modificar los parámetros de conexión sin alterar la lógica de negocio.

```
config = {
    'host':     'localhost',
    'port':     3306,
    'user':     'root',
    'password': '****',
```

```
    'database': 'clef_sport'
}

def conectar():
    return mysql.connector.connect(**config)
```

3.1.3 Operación 1 — Historial de Cliente

Llama al procedimiento almacenado sp_historial_cliente pasando la cédula como parámetro. Retorna todas las compras del cliente con detalle de productos, fechas, cantidades y subtotales, ordenadas por fecha descendente.

```
def historial_cliente(cedula):
    conn = conectar()
    cursor = conn.cursor(dictionary=True)
    try:
        cursor.callproc('sp_historial_cliente', [cedula])
        for result in cursor.stored_results():
            filas = result.fetchall()
            for f in filas:
                print(f"Venta #{f['id_venta']} | {f['fecha']} | ")
    except mysql.connector.Error as e:
        print(f"Error: {e.msg}")
    finally:
        cursor.close()
        conn.close()
```

3.1.4 Operación 2 — Registrar Venta

Llama al procedimiento sp_registrar_venta con seis parámetros. El SP valida el stock disponible, crea la venta, inserta el detalle y descuenta el inventario dentro de una transacción. La aplicación ejecuta conn.commit() explícito y conn.rollback() ante cualquier excepción.

```
def registrar_venta(id_cliente, id_usuario, fecha, metodo_pago, id_producto,
cantidad):
    conn = conectar()
    cursor = conn.cursor()
    try:
        cursor.callproc('sp_registrar_venta',
            [id_cliente, id_usuario, fecha, metodo_pago, id_producto, cantidad])
        conn.commit()
        print('Venta registrada correctamente.')
    except mysql.connector.Error as e:
        conn.rollback()
        print(f'Error al registrar venta: {e.msg}')
    finally:
        cursor.close()
        conn.close()
```

3.1.5 Operación 3 — Reporte de Ventas por Período

Llama al procedimiento sp_reporte_ventas_periodo con dos fechas. Retorna todas las ventas del período con su detalle. La aplicación acumula los subtotales y muestra el total general al final del reporte.

```
def reporte_ventas(fecha_inicio, fecha_fin):
    conn = conectar()
    cursor = conn.cursor(dictionary=True)
    try:
        cursor.callproc('sp_reporte_ventas_periodo', [fecha_inicio, fecha_fin])
        total_general = 0
        for result in cursor.stored_results():
            filas = result.fetchall()
            for f in filas:
                total_general += f['subtotal']
        print(f'TOTAL GENERAL: ${total_general:,.0f}')
    finally:
        cursor.close()
        conn.close()
```

3.1.6 Buenas Prácticas Implementadas

Práctica	Implementación
Sin SQL directo	Toda la lógica pasa por procedimientos almacenados. No existe ninguna instrucción SQL en el código Python.
Credenciales separadas	El diccionario config centraliza host, puerto, usuario y contraseña, separados de la lógica.
Manejo de errores	Bloque try/except en cada función captura mysql.connector.Error y muestra el mensaje al usuario.
Cierre de conexiones	Bloque finally garantiza cursor.close() y conn.close() independientemente del resultado.
COMMIT explícito	Las operaciones de escritura ejecutan conn.commit() solo si no ocurrió ningún error.
ROLLBACK automático	El bloque except ejecuta conn.rollback() para deshacer cambios parciales ante cualquier fallo.

3.2 — Inventario Completo de Objetos de la Base de Datos

3.2.1 Tablas

La base de datos clef_sport está compuesta por 11 tablas organizadas en tres grupos: tablas maestras, tablas transaccionales y tabla de auditoría.

Tabla	Registros	Descripción
marca	8	Nike, Adidas, Jordan, New Balance, Puma, Vans, Converse, Reebok
categoria	6	Running, Basketball, Casual, Skate, Coleccionable, Training

Tabla	Registros	Descripción
proveedor	8	Distribuidores nacionales e internacionales (Colombia, EE.UU., Alemania, México)
usuario	6	1 Administrador + 5 Vendedores con contraseñas encriptadas y rol ENUM
cliente	50	Datos personales, cédula UNIQUE, teléfono, email y dirección
producto	50	Zapatillas con talla, color, precio_venta, precio_costo y stock. FK a marca y categoría
compra	50	Encabezado de ingreso de mercancía. FK a proveedor y usuario
detalle_compra	50	Líneas de cada compra: id_compra, id_producto, cantidad, precio_unitario
venta	50	Encabezado de transacción: fecha, total, metodo_pago, estado. FK a cliente y usuario
detalle_venta	50	Líneas de cada venta: id_venta, id_producto, cantidad, precio_unitario, subtotal
auditoria_stock	Auto	Registro automático de cambios de stock: accion, stock_anterior, stock_nuevo, fecha DATETIME

3.2.2 Procedimientos Almacenados

Procedimiento	Parámetros de entrada	Función
sp_registrar_venta	id_cliente, id_usuario, fecha, metodo_pago, id_producto, cantidad	FOR UPDATE → valida stock → INSERT venta → INSERT detalle → UPDATE stock. Con transacción y ROLLBACK.
sp_historial_cliente	cedula VARCHAR(20)	JOIN venta + detalle_venta + producto filtrando por cédula. ORDER BY fecha DESC.
sp_registrar_compra	id_proveedor, id_usuario, fecha, id_producto, cantidad, precio_unitario	INSERT compra → INSERT detalle_compra → UPDATE stock + cantidad. Con transacción.
sp_anular_venta	id_venta INT	UPDATE venta SET estado='Anulada' → restaura stock del producto del detalle. Con transacción.
sp_reporte_ventas_periodo	fecha_inicio DATE, fecha_fin DATE	JOIN venta + cliente + detalle_venta + producto WHERE fecha BETWEEN. ORDER BY fecha DESC.

3.2.3 Triggers

Trigger	Tipo	Lógica
trg_auditoria_stock	AFTER UPDATE ON producto	Si OLD.stock <> NEW.stock, inserta en auditoria_stock con id_producto, accion='ACTUALIZACION', stock_anterior, stock_nuevo y CONCAT del nombre del producto.
trg_validar_stock	BEFORE INSERT ON detalle_venta	SELECT stock FROM producto WHERE id_producto = NEW.id_producto. Si stock < NEW.cantidad → SIGNAL SQLSTATE '45000': Error: Stock insuficiente para el producto.
trg_total_venta	AFTER INSERT ON detalle_venta	UPDATE venta SET total = (SELECT SUM(subtotal) FROM detalle_venta WHERE id_venta = NEW.id_venta). Recalcula el total automáticamente.

3.2.4 Índices

Índice	Tipo	Columnas	Consulta optimizada
idx_cliente_cedula	Simple	cliente(cedula)	sp_historial_cliente
idx_prod_marca_cat	Compuesto	producto(id_marca, id_categoria)	Catálogo filtrado por marca y categoría
idx_venta_fecha	Rango	venta(fecha)	sp_reporte_ventas_periodo
idx_prod_stock	Rango	producto(stock)	Alertas de stock bajo (stock < 5)
idx_dv_prod_covering	Covering	detalle_venta(id_producto, cantidad, subtotal)	Ranking de productos vendidos

3.3 — Análisis de Rendimiento con EXPLAIN

Se analizaron cinco consultas críticas del negocio antes y después de crear los índices correspondientes. El análisis se realizó usando la instrucción EXPLAIN en MySQL 8.0, que reporta el tipo de acceso (type), el número estimado de filas revisadas (rows) y el uso de índice (Extra).

3.3.1 Consulta C1 — Historial por cédula

```
EXPLAIN SELECT * FROM cliente WHERE cedula = '1001234561';
```

Parámetro	Sin índice	Con idx_cliente_cedula
type	ALL (full scan)	ref
rows	50	1
Extra	Using where	Using index condition

El índice simple sobre cedula reduce el escaneo de 50 filas a 1, convirtiendo el full scan en una búsqueda directa por referencia.

3.3.2 Consulta C2 — Catálogo por marca y categoría

```
EXPLAIN SELECT * FROM producto WHERE id_marca = 1 AND id_categoria = 5;
```

Parámetro	Sin índice	Con idx_prod_marca_cat
type	ALL	ref
rows	50	~8

El índice compuesto (id_marca, id_categoria) permite filtrar simultáneamente por ambas columnas, reduciendo las filas revisadas de 50 a aproximadamente 8.

3.3.3 Consulta C3 — Ventas por rango de fechas

```
EXPLAIN SELECT * FROM venta WHERE fecha BETWEEN '2025-08-01' AND '2025-11-30';
```

Parámetro	Sin índice	Con idx_venta_fecha
type	ALL	range
rows	50	~30
Extra	Using where	Using index condition

El índice de rango sobre fecha evita el full scan y permite localizar directamente las filas dentro del intervalo solicitado.

3.3.4 Consulta C4 — Productos con stock bajo

```
EXPLAIN SELECT * FROM producto WHERE stock < 5;
```

Parámetro	Sin índice	Con idx_prod_stock
type	ALL	range
rows	50	~3

Con el índice sobre stock, MySQL localiza directamente los productos con existencias críticas sin revisar el catálogo completo.

3.3.5 Consulta C5 — Ranking de productos vendidos (Covering Index)

```
EXPLAIN SELECT id_producto, SUM(cantidad), SUM(subtotal)
FROM detalle_venta GROUP BY id_producto ORDER BY SUM(subtotal) DESC;
```

Parámetro	Sin índice	Con idx_dv_prod_covering
type	ALL	index
Extra	Using filesort	Using index

El índice covering incluye todas las columnas necesarias (id_producto, cantidad, subtotal), por lo que MySQL resuelve la consulta completamente desde el índice sin acceder a la tabla de datos.

3.4 — Transacciones y Concurrencia

3.4.1 Nivel de Aislamiento

Se configuró el nivel de aislamiento REPEATABLE READ, que es el predeterminado en InnoDB/MySQL. Esta elección responde a la naturaleza del negocio: Clef Sport comercializa ediciones limitadas con stock reducido, donde dos vendedores podrían intentar vender el mismo par simultáneamente.

REPEATABLE READ garantiza que todas las lecturas dentro de una misma transacción devuelvan los mismos datos, evitando lecturas no repetibles. Se descartó READ COMMITTED porque permitiría que una transacción vea cambios de otras transacciones aún no confirmadas, lo que podría resultar en la venta de más unidades de las existentes.

Como complemento, sp_registrar_venta utiliza SELECT ... FOR UPDATE al consultar el stock del producto, lo que bloquea la fila durante el ciclo completo de la transacción e impide que otra sesión lea o modifique ese registro hasta el COMMIT.

```
SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SELECT @@transaction_isolation; -- Resultado: REPEATABLE-READ
```

3.4.2 Transacción 1 — Venta Exitosa

Demuestra el ciclo completo de una venta con bloqueo optimista, validación, inserción y confirmación.

```
START TRANSACTION;
    SELECT id_producto, stock FROM producto
    WHERE id_producto = 1 FOR UPDATE;
    -- El trigger trg_validar_stock verifica el stock antes del INSERT
    INSERT INTO venta (id_cliente, id_usuario, fecha, total, metodo_pago, estado)
    VALUES (3, 2, CURDATE(), 950000, 'Transferencia', 'Completada');
    INSERT INTO detalle_venta (id_venta, id_producto, cantidad, precio_unitario,
subtotal)
    VALUES (LAST_INSERT_ID(), 1, 1, 950000, 950000);
    -- trg_total_venta recalcula el total de la venta automáticamente
    UPDATE producto SET stock = stock - 1 WHERE id_producto = 1;
    -- trg_auditoria_stock registra el cambio en auditoria_stock
COMMIT;
```

3.4.3 Transacción 2 — Fallo y ROLLBACK

Demuestra la atomicidad del sistema: si cualquier instrucción falla, ningún cambio persiste en la base de datos.

```
START TRANSACTION;
    INSERT INTO venta (id_cliente, id_usuario, fecha, total, metodo_pago, estado)
    VALUES (7, 3, CURDATE(), 1500000, 'Efectivo', 'Completada');
    -- trg_validar_stock rechaza 9999 unidades (stock real = 30)
    INSERT INTO detalle_venta (..., 9999, ...) -- ERROR 45000
ROLLBACK;
-- La venta insertada NO existe en la BD. Stock intacto.
```

El EXIT HANDLER del procedimiento captura la excepción SQLEXCEPTION, ejecuta ROLLBACK y relanza el error al llamador mediante RESIGNAL.

3.4.4 Transacción 3 — SAVEPOINT (Rollback Parcial)

Demuestra que es posible deshacer una parte de una transacción conservando el trabajo ya confirmado hasta el SAVEPOINT.

```
START TRANSACTION;
    INSERT INTO compra (...) VALUES (1, 1, CURDATE(), 12000000, 'Recibida');
    SAVEPOINT sp_encabezado_ok;
    INSERT INTO detalle_compra (...) VALUES (..., 1, 10, 600000); -- OK
    UPDATE producto SET stock = stock + 10 WHERE id_producto = 1;
    SAVEPOINT sp_primer_detalle_ok;
    -- Segundo detalle con FK inválida → error
    ROLLBACK TO SAVEPOINT sp_primer_detalle_ok;
COMMIT; -- compra + primer detalle persisten; segundo no
```

3.4.5 Simulación de Deadlock

Se simuló una situación de deadlock con dos sesiones que intentan bloquear los mismos productos en orden inverso.

Paso	Sesión A	Sesión B
1	START TRANSACTION; SELECT ... id=1 FOR UPDATE	(espera)
2	(espera)	START TRANSACTION; SELECT ... id=2 FOR UPDATE
3	UPDATE ... id=2 → espera lock de B	—
4	Continúa → COMMIT	UPDATE ... id=1 → DEADLOCK → ERROR 1213

InnoDB detecta el deadlock automáticamente y aborta la transacción con mayor costo (Sesión B), permitiendo que Sesión A complete su operación. La prevención se implementa accediendo siempre a los productos en el mismo orden (ORDER BY id_producto ASC) y manteniendo las transacciones lo más cortas posible.

3.5 — Estructura del Repositorio

El proyecto se organizó en tres carpetas principales según el tipo de artefacto entregado.

```
CLefSport_corte3/
├── app/
│   ├── app.py              # Código principal de la aplicación Python
│   └── README.md           # Instrucciones de instalación y ejecución
├── scripts/
│   ├── clef_sport_completo.sql  # Dump completo de la BD (DDL + DML)
│   └── triggers.sql             # Triggers e índices
└── informe/
    ├── ClefSport_Corte1_DEFINITIVO.pdf
```

```
    ├── ClefSport_Corte2_Informe.pdf
    └── ClefSport_Corte3_Informe.pdf
```

Archivo	Contenido
app.py	Aplicación de consola Python con menú interactivo y 3 operaciones usando SPs
README.md	Instrucciones de instalación, configuración y ejemplos de uso
clef_sport_completo.sql	Script completo: CREATE DATABASE, tablas, datos de prueba y SPs
triggers.sql	3 triggers (trg_auditoria_stock, trg_validar_stock, trg_total_venta) y 5 índices

3.6 — Lecciones Aprendidas y Conclusiones

3.6.1 Lecciones Aprendidas

Durante el desarrollo del proyecto Clef Sport a lo largo de los tres cortes se identificaron las siguientes lecciones:

Los triggers son la primera línea de defensa de la integridad

La validación de stock en trg_validar_stock demostró que es más robusto validar en la base de datos que en la aplicación. Si la lógica de validación estuviera solo en Python, cualquier acceso directo a la BD (desde Workbench u otra app) podría saltarla. Los triggers ejecutan siempre, independientemente del cliente.

REPEATABLE READ + FOR UPDATE es la combinación correcta para e-commerce

Una tienda de ediciones limitadas con stock reducido necesita garantías fuertes. La combinación de nivel de aislamiento REPEATABLE READ y bloqueo explícito con SELECT ... FOR UPDATE elimina la posibilidad de double-selling, que sería el error más costoso para el negocio.

Los procedimientos almacenados simplifican el código de aplicación

Al centralizar toda la lógica en SPs, el código Python quedó muy limpio: solo recibe datos del usuario, llama al SP y muestra el resultado. Esto facilita el mantenimiento porque un cambio en la lógica de negocio solo requiere modificar el SP, sin tocar la aplicación.

Los índices covering eliminan accesos a tabla

El índice idx_dv_prod_covering demostró que cuando todas las columnas de una consulta están en el índice, MySQL no necesita leer la tabla de datos. Esto es especialmente valioso en tablas de detalle que crecen rápidamente.

El manejo de errores debe ser exhaustivo

Cada función de la app Python tiene su propio bloque try/except/finally. La experiencia mostró que sin el bloque finally, una excepción dejaría conexiones abiertas, lo que eventualmente agotaría el pool de conexiones de MySQL.

3.6.2 Conclusiones

El proyecto Clef Sport logró implementar una base de datos relacional completa en MySQL 8.0 que centraliza todos los procesos críticos de una tienda de calzado deportivo: gestión de inventario, registro de ventas, control de compras a proveedores y auditoría automática de cambios.

La integración con Python demostró que una arquitectura basada en procedimientos almacenados produce aplicaciones más seguras, mantenibles y eficientes que el acceso directo con SQL dinámico. La separación de responsabilidades entre la base de datos (lógica de negocio) y la aplicación (interfaz de usuario) es una práctica que el proyecto aplica consistentemente.

Los tres cortes del proyecto cubrieron progresivamente los pilares de un sistema de base de datos empresarial: diseño y normalización (Corte 1), optimización y lógica avanzada (Corte 2), e integración con aplicaciones externas (Corte 3). El resultado es un sistema funcional, documentado y listo para escalar.

Referencias —

MySQL. (2024). MySQL 8.0 Reference Manual. Oracle Corporation. https://dev.mysql.com/doc/refman/8.0/en/

Python Software Foundation. (2024). Python 3.13 Documentation. https://docs.python.org/3/

mysql-connector-python. (2024). MySQL Connector/Python Developer Guide. Oracle Corporation. https://dev.mysql.com/doc/connector-python/en/

Elmasri, R., & Navathe, S. (2016). Fundamentals of Database Systems (7th ed.). Pearson.

Silberschatz, A., Korth, H., & Sudarshan, S. (2019). Database System Concepts (7th ed.). McGraw-Hill.